

本章目标

从 malloc/free 出发,理解 C++ 的 new/delete 与对象生命周期;掌握三大件与五法则的深拷贝纪律;吃透 RAI 的资源管理思想;熟练运用 unique_ptr、shared_ptr、weak_ptr 三种智能指针,识别并修复循环引用。本章是 C 程序员向 C++ 转型最重要的一章。

前置知识

熟练掌握 C 的 malloc/free、指针与结构体;了解第4章的引用、成员函数与构造函数概念。本章代码基于 C++17,使用智能指针必须 include <memory>。

\$5.1 new/delete 与对象生命周期	\$5.2 三大件与五法则	\$5.3 RAII 资源管理
\$5.4 unique_ptr 独占所有权	\$5.5 shared_ptr 与 weak_ptr	\$5.6 常见内存错误与工具

\$5.1 从 malloc/free 到 new/delete

C 程序员用 malloc/free 管理堆内存:malloc 只分配一片字节,返回 void*,需要强转成目标类型;free 只回收字节,不做任何清理。C++ 对象有构造函数和析构函数,分配内存的同时必须构造对象、释放内存前必须先析构对象,所以 new/delete 把两步合并成一次操作:new 分配内存后调用构造函数,delete 先调用析构函数再释放内存。这是一切内存管理的起点。

配套规则:new 分配的对象用 delete 释放,new[] 分配的数组必须用 delete[] 释放,混用是未定义行为。new 失败时抛出 std::bad_alloc 异常,而不是像 malloc 那样返回 NULL——这是 C++ 默认的失败模型,如果你需要老式的不抛异常版本,可以用 new(std::nothrow),它会返回空指针。new[] 在数组开头额外记录元素个数,delete[] 靠它逐个调用析构函数,这也是二者必须配对的原因。

C 的写法	C++ 的写法	说明
int* p = malloc(sizeof(int) * 10);	int* p = new int[10];	C 手算字节数并强转,C++ 类型安全、自动计数
free(p);	delete[] p;	C 只回收字节,C++ 先析构每个元素再释放
if (!p) { 处理失败 }	new 抛 std::bad_alloc	C 靠返回值判断,C++ 走异常路径
结构体需要手动 memset 初始化	new 自动调用构造函数	C++ 对象保证构造完成,不可用半初始化状态

清单 5-1 new/delete 与 new[]/delete[]

```

#include <iostream>
#include <string>

class Timer {
public:
    explicit Timer(const std::string& n) : name(n) {
        std::cout << "构造:" << name << "\n";
    }
    ~Timer() { std::cout << "析构:" << name << "\n"; } // 释放前必被调用
private:
    std::string name;
};

int main() {
    Timer* t = new Timer("单对象"); // 分配 + 构造
    delete t;                       // 析构 + 释放,配对

    Timer* arr = new Timer[3]{      // new[]:每个元素构造
        Timer("甲"), Timer("乙"), Timer("丙")};
    delete[] arr;                  // 逐个析构后再释放

    int* raw = new (std::nothrow) int(42); // 失败返回空指针,不抛异常
    if (raw) {
        std::cout << *raw << "\n";
        delete raw;
    }
    return 0;
}

```

⚠ 易错点 1:new 与 delete 不配对

new 用 delete,new[] 用 delete[],漏写或混写都是错误:漏写泄漏内存,混写是未定义行为。现代 C++ 的答案不是"记得配对",而是根本不手写 new/delete——用容器与智能指针(\$5.4),让析构自动完成配对。

§5.2 三大件与五法则

一个类如果手动管理资源(比如持有一个 new 出来的指针成员),就必须同时提供拷贝构造、拷贝赋值和析构这三个函数,合称三大件:拷贝构造负责用另一个对象初始化自己,拷贝赋值负责把自己的内容替换成另一个对象的内容,析构负责释放自己占用的资源。编译器默认生成的版本对内置类型逐位拷贝——对指针成员来说,默认拷贝只是复制指针值,两个对象将指向同一块内存,析构时先后释放两次,这就是双重释放,是 C++ 最经典的内存崩溃之一。

C++11 又加入了移动构造与移动赋值,三大件扩成五法则:移动语义允许"把资源搬走而不是复制"。移动构造的要点是接管源对象的指针,然后把源指针置空,这样源对象析构时无事可做,不会释放已被接管的内存。移动操作必须声明 noexcept,否则 vector 扩容时无法放心使用移动。经验法则:写了析构函数就基本可以断定需要五件套,尤其是持有裸指针、文件句柄等资源的类。

C 的写法	C++ 的写法	说明
struct 逐位拷贝,共享指针成员	手写深拷贝,各自持有副本	C 默认浅拷贝,双重释放隐患;C++ 用五法则修正
memcpy 复制结构体	拷贝构造 / 拷贝赋值	C 不调用任何函数,C++ 构造与赋值语义完整
无法表达"搬走资源"	移动构造,源对象置空	C++ 转移所有权,大对象零拷贝传递

```

#include <iostream>
#include <string>
#include <utility>

class NameHolder {
public:
    explicit NameHolder(const std::string& n)
        : p(new std::string(n)) {} // new:分配 + 构造

    NameHolder(const NameHolder& o) // 三大件:拷贝构造
        : p(new std::string(*o.p)) {} // 深拷贝,绝不共享指针

    NameHolder& operator=(const NameHolder& o) { // 三大件:拷贝赋值
        if (this != &o) { // 自赋值防护
            *p = *o.p; // 复用已有内存
        }
        return *this;
    }

    NameHolder(NameHolder&& o) noexcept // 五法则:移动构造
        : p(o.p) { o.p = nullptr; } // 接管指针并置空源

    NameHolder& operator=(NameHolder&& o) noexcept { // 移动赋值
        if (this != &o) {
            delete p; // 先释放自己的
            p = o.p;
            o.p = nullptr;
        }
        return *this;
    }

    ~NameHolder() { delete p; } // 三大件:析构

    const std::string& get() const { return *p; }
private:
    std::string* p;
};

int main() {
    NameHolder a("原始");
    NameHolder b = a; // 拷贝构造:两块独立内存
    NameHolder c(std::move(a)); // 移动构造:a 被置空,资源归 c
    b = c; // 拷贝赋值
    std::cout << b.get() << " " << c.get() << "\n";
    return 0; // 三个对象各自释放,无双重释放
}

```

⚠ 易错点 2:忘记三大件导致的浅拷贝

类里出现裸指针成员,却依赖编译器默认的逐位拷贝,两个对象就会共享同一块内存:改一个影响另一个,析构时双双释放同一块内存,直接崩溃。诊断口诀:出现了 delete 却不见拷贝构造,先怀疑浅拷贝。

§5.3 RAII:把资源绑定到对象生命周期

RAII(Resource Acquisition Is Initialization,资源获取即初始化)是 C++ 最核心的工程思想:在构造函数里获取资源,在析构函数里释放资源,资源与对象的生命周期严格一致。对象创建,资源就绪;对象销毁,资源自动回收——无论正常返回还是抛出异常,析构函数都保证执行,资源绝不泄漏。

对比 C 的手工管理:文件要 fopen 后记得 fclose,而且每一条出错路径都得写一遍 fclose,漏一条就泄漏。RAII 把"记得释放"变成"必然释放":C++ 中栈对象离开作用域必然调用析构函数,这是语言保证,不需要程序员记忆。标准库就是 RAII 的样板:std::string、std::vector 自己分配内存自己释放,你从来不需要对它们调用 free;文件流 ifstream 关闭时自动关闭文件。写自定义类时,凡是"打开就要关闭、获取就要释放"的资源——文件、锁、套接字、数据库连接——都应包装成 RAII 类。

C 的写法	C++ 的写法	说明
FILE* f = fopen(...); ... fclose(f);	ofstream f(path); 结束自动关闭	C 每路径手工释放,C++ 析构保证释放
出错路径容易漏 fclose	抛异常也执行析构	RAII 异常安全,无遗漏路径
锁:lock/unlock 成对手写	std::lock_guard 自动解锁	C++ 锁对象即解锁,死锁少一半

清单 5-3 RAII 包装 C 资源

```

#include <cstdio>
#include <string>

class FileGuard {                                // RAII:构造获取,析构释放
public:
    explicit FileGuard(const std::string& path, const char* mode)
        : f(fopen(path.c_str(), mode)) {
        if (!f) throw std::runtime_error("打开失败: " + path);
    }
    ~FileGuard() {
        if (f) {
            fclose(f);                        // 无论正常还是异常返回,必被执行
            f = nullptr;
        }
    }
    FileGuard(const FileGuard&) = delete;        // 不可复制:句柄唯一
    FileGuard& operator=(const FileGuard&) = delete;
    void write(const std::string& s) { fputs(s.c_str(), f); }
private:
    FILE* f;
};

void logAll(const std::string& path) {
    FileGuard log(path, "w");                // 构造:打开文件
    for (int i = 0; i < 3; ++i) {
        if (i == 2) throw std::runtime_error("中途出错");
        log.write("第 " + std::to_string(i) + " 行\n");
    }
}                                           // 析构:即使抛异常,文件也关闭

int main() {
    try {
        logAll("run.log");
    } catch (const std::exception& e) {
        std::printf("捕获:%s\n", e.what());
    }
    return 0;
}

```

§5.4 unique_ptr:独占所有权

unique_ptr 是独占所有权的智能指针:同一时刻只有一个指针拥有这块内存,不允许拷贝,只能移动——移动即所有权转移。它在析构时自动 delete 管理的对象,把"手工配对"变成"自动释放"。创建用 std::make_unique(第一个参数是构造参数,自动推导类型),C++14 起可用。unique_ptr 可以持有数组,声明为 unique_ptr<int[]>,析构时自动用 delete[]。

unique_ptr 的价值不只是防泄漏,更是把所有权写进类型:函数参数用 const unique_ptr& 表示"只借用",返回值用 unique_ptr 表示"所有权移交",容器里放 unique_ptr 表示"容器独占这些元素"。裸指针可以随时被拷贝、被遗忘,所有权说不清楚;unique_ptr 让"谁拥有、谁释放"在编译期就确定下来。

清单 5-5 容器中的 unique_ptr

```

#include <memory>
#include <vector>

std::vector<std::unique_ptr<int>> bucket;    // 容器独占元素

void fill(int n) {
    for (int i = 0; i < n; ++i) {
        bucket.push_back(std::make_unique<int>(i));
    }
}    // 返回后元素仍存活,所有权在容器

int main() {
    fill(3);
    for (const auto& p : bucket) std::cout << *p << " ";
    bucket.clear();    // 清空时逐个 delete,无需手工释放
    return 0;
}

```

⚠ 易错点 3:裸指针的悬挂与重复释放

裸指针在 delete 之后仍然保存着地址,继续使用就是悬空指针,未定义行为;同一块内存被 delete 两次就是重复释放,直接崩溃。智能指针从机制上消灭这两类错误:unique_ptr 不能拷贝所以只有一个释放者,shared_ptr 只有计数归零才释放。

§5.5 shared_ptr 与 weak_ptr:共享所有权

shared_ptr 是共享所有权:多个 shared_ptr 可以指向同一对象,每个都保存一个引用计数,每拷贝一次计数加一,每析构一个计数减一,计数归零才真正释放对象。计数操作是原子的,多个线程同时拷贝与销毁是安全的。创建用 std::make_shared,它把控制块和对象一次性分配,比先 new 再包 shared_ptr 少一次分配。代价是引用计数本身有开销,以及可能产生循环引用。

循环引用是 shared_ptr 最著名的陷阱:两个对象互相持有对方的 shared_ptr,彼此的计数都大于零,谁也不会释放,内存悄然泄漏。解决办法是 weak_ptr:它是"旁观者",指向对象但不增加计数,不能直接访问,必须调用 lock() 提升为 shared_ptr,对象已销毁时 lock() 返回空指针。使用套路:单向共享用 shared_ptr,回指、缓存、观察等"不需要延长生命"的引用一律用 weak_ptr。

C 的写法	C++ 的写法	说明
裸指针传遍全程序,不知谁释放	unique_ptr:独占;shared_ptr:共享	所有权写进类型,释放点明确
手工引用计数结构体	make_shared 自动计数	计数线程安全,归零自动释放
无对应概念	weak_ptr 观察不持有	打破循环引用,避免内存泄漏

清单 5-4 三种智能指针与循环引用修复

```

#include <iostream>
#include <memory>

struct Worker;
struct Team {                                // 团队:shared_ptr 持有成员
    std::shared_ptr<Worker> member;
    ~Team() { std::cout << "团队析构\n"; }
};

struct Worker {                              // 员工:weak_ptr 回指团队,不增计数
    std::weak_ptr<Team> team;
    ~Worker() { std::cout << "员工析构\n"; }
};

int main() {
    std::cout << "--- 正常共享 ---\n";
    {
        auto t = std::make_shared<Team>();
        t->member = std::make_shared<Worker>();
        t->member->team = t;                // weak_ptr 赋值,不增加计数
        auto sp = t->member->team.lock();   // 提升为 shared_ptr
        if (sp) std::cout << "回指成功\n";
    }                                       // t 与 member 双双正常析构

    std::cout << "--- 独占所有权 ---\n";
    auto up = std::make_unique<int>(99);   // unique_ptr 独占
    std::unique_ptr<int> up2 = std::move(up); // 移动转移所有权
    // std::unique_ptr<int> up3 = up2;       // 错误:不允许拷贝
    std::cout << *up2 << "\n";             // 作用域结束自动 delete
    return 0;
}

```

⚠ 易错点 4:shared_ptr 循环引用悄然泄漏

Team 持 Worker,Worker 又持 Team,两边计数互相兜底,永远不为零,两个对象一起泄漏。工具类(如 AddressSanitizer 的报告)能看出"泄漏的是互相引用的两块内存",但根治靠设计:区分"拥有"与"观察",观察一律用 weak_ptr。

§5.6 内存布局与常见错误排查

C++ 程序的内存分四块:栈区存放局部变量,自动分配自动回收,速度快但容量小;堆区由 new/malloc 分配,程序员或智能指针负责回收;静态区存放全局变量与静态变量,整个程序生命周期都在;常量区存放字符串字面量等只读数据。栈上对象离开作用域必然析构,堆上对象只有释放才析构——这决定了"对象活了多久"的全部问题。

排查内存错误用三件武器:AddressSanitizer(ASan)在编译时加 -fsanitize=address,运行时报出越界、悬空、泄漏的精确行号;valgrind 适合没有 ASan 的环境,慢但全面;泄漏检测(LeakSanitizer)专门报告未释放的内存。现代工程默认开启 ASan,把它当成编译器自带的体检仪。性能方面,堆分配比栈慢两个数量级,热点路径避免频繁 new 小对象,优先复用对象或用容器一次性分配。

内存管理速查表

概念	语法示例	要点
new/delete	int* p = new int(1); delete p;	分配+构造 / 析构+释放,必须配对
new[]/delete[]	int* a = new int[10]; delete[] a;	数组用数组版本,混用是未定义行为
不抛异常分配	new (std::nothrow) int(1)	失败返回空指针,适合兼容老代码
三大件	拷贝构造/拷贝赋值/析构	手动管理资源必须三件齐全
五法则	三大件 + 移动构造/移动赋值	移动置空源指针,声明 noexcept
深拷贝	new 一份副本给新对象	杜绝浅拷贝共享指针
RAII	构造获取、析构释放	资源绑定对象生命周期,异常安全
unique_ptr	make_unique<T>(args)	独占所有权,只可移动不可拷贝
shared_ptr	make_shared<T>(args)	引用计数,归零释放,计数线程安全
weak_ptr	w.lock() 判空后使用	不增计数,破循环引用
内存泄漏	new 后无人 delete	ASan / valgrind 定位泄漏点
悬空指针	delete 后继续使用	未定义行为,智能指针根治
双重释放	同一内存 delete 两次	浅拷贝与裸指针的经典事故
make_unique 数组	make_unique<int[]>(10)	自动配对 delete[],C++14 起

自测题

- Q 1. new 与 malloc 的本质区别是什么?new[] 与 delete[] 为什么要配对?**
- Q 2. 为什么带指针成员类不能依赖默认拷贝?深拷贝与浅拷贝有何区别?**
- Q 3. 移动构造为什么要把源指针置空?移动函数为什么要声明 noexcept?**
- Q 4. shared_ptr 循环引用为什么泄漏?weak_ptr 是如何解决的?**
- Q 5. RAII 为什么能做到异常安全?请举一个"不用 RAII 就会泄漏"的场景。**

✓ 参考答案

- malloc 只分配字节、返回 void*、失败返回 NULL;new 分配后调用构造函数、类型安全、失败抛 std::bad_alloc。new[] 在数组头记录元素个数,delete[] 靠它逐个析构;与 delete 混用会错误解析内存布局,未定义行为。
- 默认拷贝逐位复制指针,两个对象指向同一内存,一方修改影响另一方,析构时双重释放。深拷贝为每个新对象分配独立副本,各自管理互不干扰,是手动管理内存的必修课。
- 移动是"搬走"而非"复制":接管源对象的指针后必须把源指针置空,源析构时才不会二次释放。声明 noexcept 让容器(如 vector 扩容)敢用移动而非退回拷贝,否则性能与正确性都打折。
- A 持 B、B 持 A,两者计数都因对方存在而大于零,永远不归零,对象成对泄漏。weak_ptr 不增加计数,只做观察;需要访问时 lock() 提升为 shared_ptr,对象已销毁则得到空指针。
- 栈对象离开作用域必然调用析构函数,这是语言保证,不依赖程序员记忆;无论函数正常返回还是中途抛异常,析构都执行。典型场景:打开文件后中间代码抛异常,若不用 RAII 包装,文件将永远不关闭。

实验:员工与部门的内存安全示例

本实验用一个"员工链表 + 部门共享 + 日志 RAII"的完整程序,串联本章全部知识点:new/delete、三大件、五法则的移动、RAII、unique_ptr、shared_ptr、weak_ptr 与循环引用修复、内存泄漏自检。程序构造三名员工组成链表,员工通过 weak_ptr 回指部门,最后用静态计数验证析构完全、无泄漏。代码分三部分,覆盖点已在注释中标明。

实验 5•第一部分:五法则类(覆盖:new/delete、三大件、五法则移动)

```
#include <iostream>
#include <string>
#include <memory>
#include <cstdio>

class Employee {
public:
    Employee(const std::string& n, double s)
        : name(new std::string(n)), salary(s) { ++alive; }

    Employee(const Employee& o) // 三大件:拷贝构造
        : name(new std::string(*o.name)), salary(o.salary) { ++alive; }

    Employee& operator=(const Employee& o) { // 三大件:拷贝赋值
        if (this != &o) {
            *name = *o.name; // 深拷贝,不共享
            salary = o.salary;
        }
        return *this;
    }

    Employee(Employee&& o) noexcept // 五法则:移动构造
        : name(o.name), salary(o.salary) { o.name = nullptr; }

    Employee& operator=(Employee&& o) noexcept { // 五法则:移动赋值
        if (this != &o) {
            delete name;
            name = o.name;
            salary = o.salary;
            o.name = nullptr;
        }
        return *this;
    }

    ~Employee() { delete name; --alive; } // 三大件:析构

    const std::string& getName() const { return *name; }
    double getSalary() const { return salary; }
    void raise(double by) { salary += by; }
    static int aliveCount() { return alive; }

private:
    std::string* name; // 手动管理的堆内存
    double salary;
    inline static int alive = 0; // 存活计数:结束时为 0 即无泄漏
};
```

实验 5•第二部分:RAII 日志与智能指针(覆盖:RAII、unique_ptr、shared_ptr、weak_ptr)

```
class LogGuard {                                // 覆盖:RAII,析构自动关文件
public:
    explicit LogGuard(const char* path)
        : f(fopen(path, "w")) {
        if (!f) throw std::runtime_error("日志打开失败");
    }
    ~LogGuard() { if (f) fclose(f); }
    void log(const std::string& s) { fputs(s.c_str(), f); }
    LogGuard(const LogGuard&) = delete;
    LogGuard& operator=(const LogGuard&) = delete;
private:
    FILE* f;
};

struct Department {                             // 覆盖:shared_ptr 共享持有
    std::string name;
    int quota;
    Department(const std::string& n, int q) : name(n), quota(q) {}
};

struct EmpNode {                                // 覆盖:unique_ptr 链表节点
    Employee emp;
    std::unique_ptr<EmpNode> next;
    std::weak_ptr<Department> dept;    // 覆盖:weak_ptr 回指,不增计数
    EmpNode(Employee e, const std::shared_ptr<Department>& d)
        : emp(std::move(e)), dept(d) {}
};

std::unique_ptr<EmpNode> buildList(const std::shared_ptr<Department>& d) {
    auto head = std::make_unique<EmpNode>(Employee("张三", 8000), d);
    head->next = std::make_unique<EmpNode>(Employee("李四", 9000), d);
    head->next->next = std::make_unique<EmpNode>(Employee("王五", 7500), d);
    return head;                               // 所有权随返回值转移
}
```

实验 5•第三部分:main 验证(覆盖:weak_ptr.lock、make_shared、五法则综合、new/delete 配对)

```

int main() {
    LogGuard log("dept.log");           // RAII:结束自动关闭

    auto dept = std::make_shared<Department>("研发部", 10);
    auto list = buildList(dept);        // unique_ptr 链表

    std::shared_ptr<Department> got = list->dept.lock(); // 提升为 shared_ptr
    if (got) {
        log.log("部门: " + got->name + ", 名额 " + std::to_string(got->quota));
    }

    Employee copy = list->emp;           // 拷贝构造:深拷贝
    Employee moved = std::move(copy); // 移动构造:接管后 copy 为空
    moved.raise(500);
    log.log("调动: " + moved.getName() + " -> " + std::to_string(moved.getSalary()));

    double total = 0.0;
    for (const EmpNode* p = list.get(); p; p = p->next.get()) {
        total += p->emp.getSalary();
    }
    log.log("工资总额: " + std::to_string(total));

    int* raw = new int(42);              // 覆盖:new/delete 手工配对
    std::cout << "原始值 " << *raw << ", 存活员工 " << Employee::aliveCount() << "\n";
    delete raw;

    return 0;                            // 链表、部门、日志自动析构
    // 程序结束前打印存活员工数:应回到 0,证明无泄漏
}

```

实验覆盖点核对:new/delete(Employee 构造与析构、main 中的 raw)、三大件(拷贝构造与拷贝赋值深拷贝)、五法则(移动构造与移动赋值置空源指针)、RAII(LogGuard 自动关文件,即使抛异常)、unique_ptr(链表独占节点,自动 delete[] 与 delete)、shared_ptr(make_shared 的部门)、weak_ptr(员工回指部门不增计数,lock 提升判空)。运行程序观察析构顺序与最后的存活计数,把 LogGuard 换成裸 fopen 再故意抛异常,对比文件是否关闭,体会 RAII 的价值。

工程建议 1:默认使用智能指针与容器

新代码中裸 new 只出现在 make_unique/make_shared 里,裸 delete 根本不该出现;字符串、数组一律交给 std::string 与 std::vector。它们都是 RAII 对象,自己管理内存,你的代码里将没有一条手工释放语句,内存错误先少一半。

工程建议 2:开 ASan 跑测试

本地编译加 -fsanitize=address -fno-omit-frame-pointer,持续集成里也跑一遍;越界、悬空、泄漏都会带精确行号报出来。它是 C/C++ 工程性价比最高的内存体检工具,配合 valgrind 双保险,内存 bug 无处遁形。

本章小结

内存管理的全貌:new/delete 把分配与构造、析构与释放绑定在一起;三大件与五法则保证手动管理资源的类拷贝安全;RAII 让资源释放成为语言保证;unique_ptr、shared_ptr、weak_ptr 把所有权语义写进类型,循环引用用 weak_ptr 破解。下一章进入面向对象:类、继承与多态。